

Procura adversarial

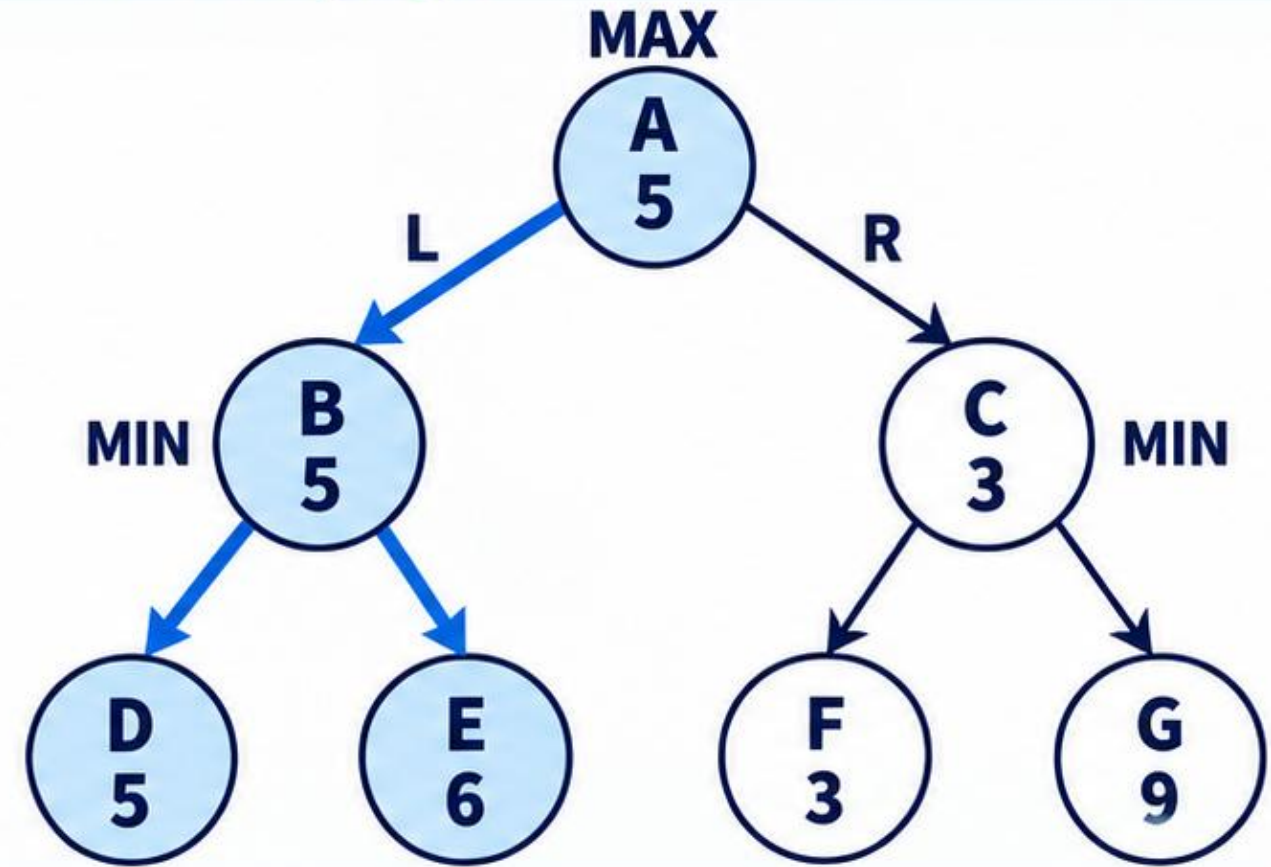
Minimax, poda alfa-beta e avaliação de estados

Escolher uma ação antecipando a melhor resposta do adversário

O que vamos aprender

- 1 Modelar estados, ações e utilidades
- 2 Executar Minimax e justificar a decisão
- 3 Aplicar a poda alfa-beta
- 4 Comparar ordenação e custo da procura
- 5 Usar avaliação heurística e verificar resultados

Árvore de jogo



$$V(\text{raiz}) = \max(\min(5, 6), \min(3, 9)) = 5$$

**Competências
a desenvolver**



Modelar

Definir regras e hipóteses



Decidir

Propagar valores MAX e MIN



Verificar

Comparar resultados e esforço

Janelas de atracação: decisões dependentes

Uma escolha altera as alternativas disponíveis para o outro operador.

Dois operadores: A e B

Escolhas alternadas

Atribuição exclusiva

Estado inicial s_0

Turno: operador A

J_1

J_2

J_3

A escolhe
 J_2

Novo estado s_1

Turno: operador B

J_1

J_3

Respostas de B

Escolher J_1

Escolher J_3

$$J_{\text{livres}}(s_1) = J_{\text{livres}}(s_0) \setminus \{J_2\} = \{J_1, J_3\}$$

Que ação escolher, considerando as respostas que podem surgir?

Hipóteses do modelo adversarial

Distinguir a motivação operacional da simplificação usada no algoritmo.

Cenário operacional

Cada operador procura reduzir o próprio tempo de espera.

Interesses diferentes não demonstram, por si só, uma relação de soma zero.



Hipótese: A = MAX; B = MIN

MAX maximiza U; MIN minimiza a mesma utilidade para MAX.

$$U_{\text{MIN}}(s) = -U_{\text{MAX}}(s)$$

Agente	Objetivo em termos de U_{MAX}	Utilidade própria
A (MAX)	Maximizar U_{MAX}	$U_{\text{MAX}}(s)$
B (MIN)	Minimizar U_{MAX}	$U_{\text{MIN}}(s) = -U_{\text{MAX}}(s)$



Dois agentes

A e B com objectivos opostos.



Turnos alternados

Agentes actuam de forma alternada.



Informação completa

Ambos conhecem o estado e as acções disponíveis.



Transições determinísticas

O próximo estado resulta deterministicamente da acção escolhida.

Modelo didático de pior caso; não descreve automaticamente o comportamento real.

Representar o estado e a transição

Guardar a informação necessária para gerar ações e interpretar o resultado.

1. Representação do estado

$$S = (J_{\text{livres}}, M_A, M_B, p)$$

2. Campos do estado

Campo	Significado
J livres	Janelas ainda disponíveis
M A	Janelas atribuídas a A
M B	Janelas atribuídas a B
p	Jogador do próximo turno

3. Exemplo: A escolhe J₂

ANTES

$$(\{J_1, J_2, J_3\}, \emptyset, \emptyset, A)$$



DEPOIS

$$(\{J_1, J_3\}, \{J_2\}, \emptyset, B)$$

J₂ sai das livres, entra em M A e o turno muda.

Interface do jogo e condições de validade

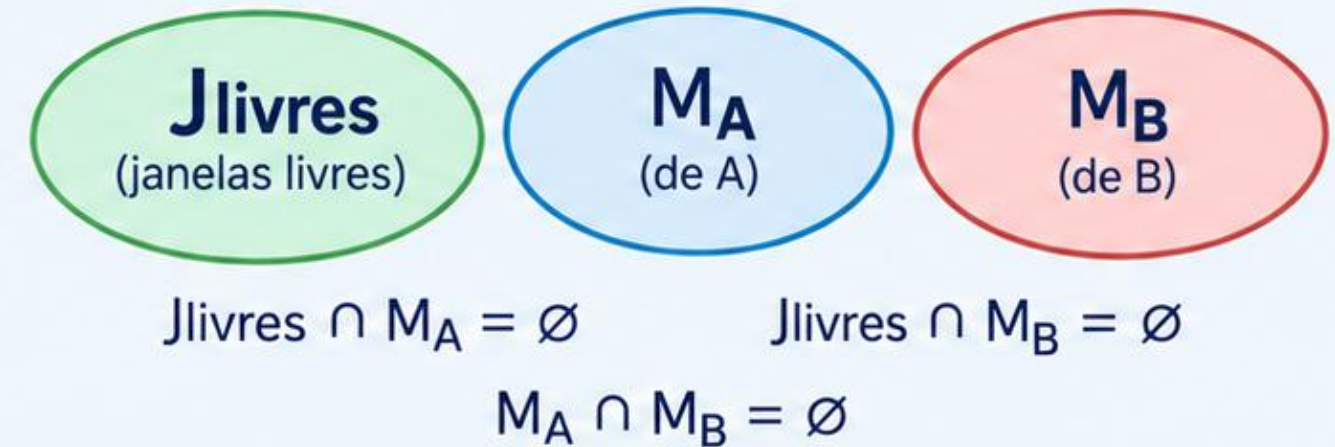
Separar as regras do problema do algoritmo de procura.

Contrato das funções

Função	Responsabilidade
jogador(s)	Indica MAX ou MIN
acoes(s)	Devolve ações legais
resultado(s, a)	Cria o estado sucessor
terminal(s)	Sem janelas livres neste modelo
U(s)	Utilidade terminal para MAX

Verificações do modelo

- A janela escolhida tem de estar livre.
- O turno deve alternar entre A e B.
- Os três conjuntos são disjuntos.



CONSERVAÇÃO DAS JANELAS

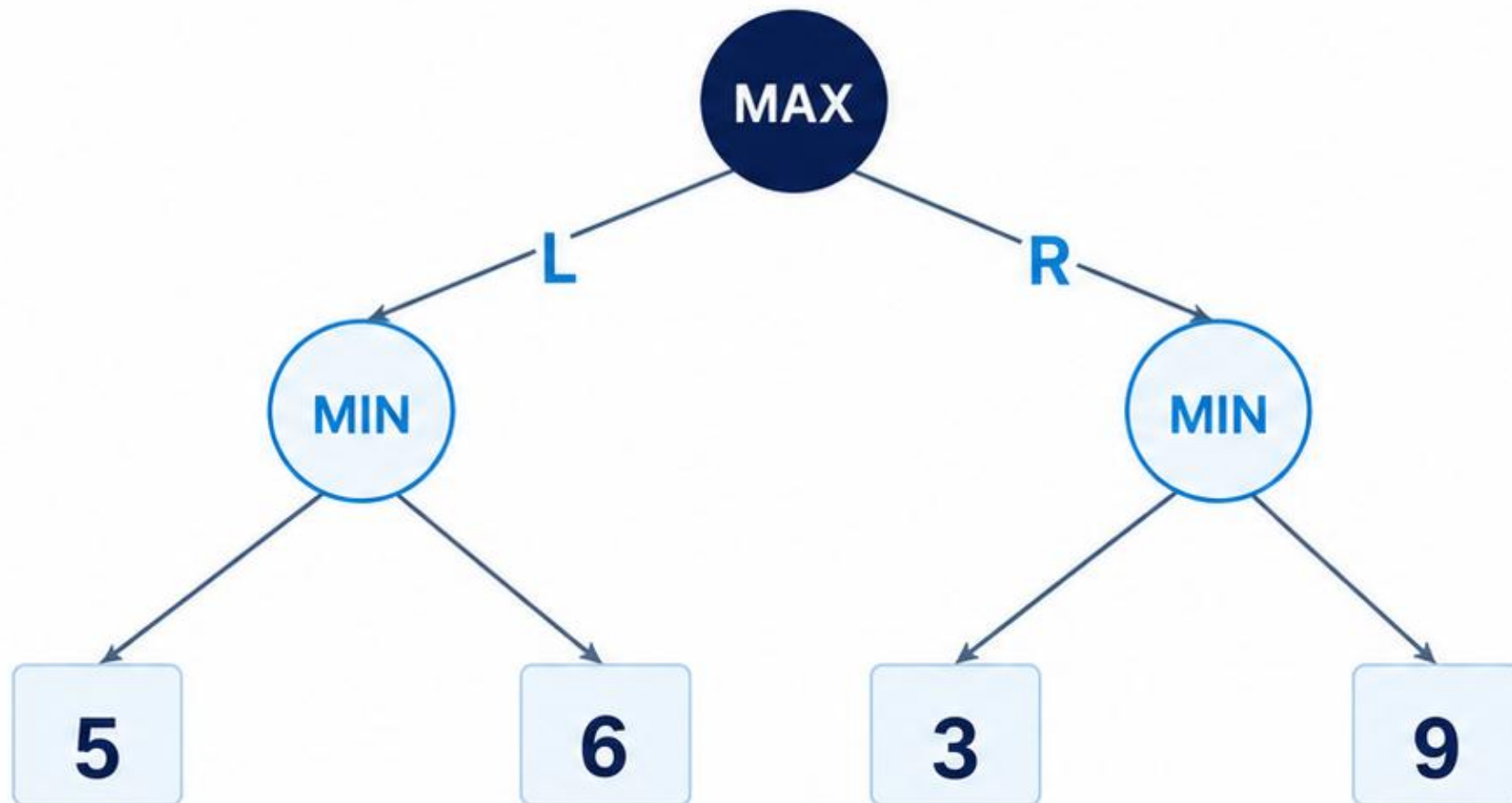
$$J_{livres} \cup M_A \cup M_B = J_0$$

J_0 : conjunto inicial. Verificações adicionais derivadas da representação proposta.

Da utilidade à árvore MAX/MIN

As ligações são ações; os valores das folhas são utilidades, não custos de arcos.

Árvore de jogo (valores de utilidade)



Uma escala para MAX

MAX escolhe valores maiores.
MIN escolhe valores menores.

Ação	Possível	Garantido
L	5 ou 6	5
R	3 ou 9	3

Nó = estado

Ligação = ação

Folha = $U(s)$

A folha 9 existe, mas MIN não é obrigado a escolher essa resposta.

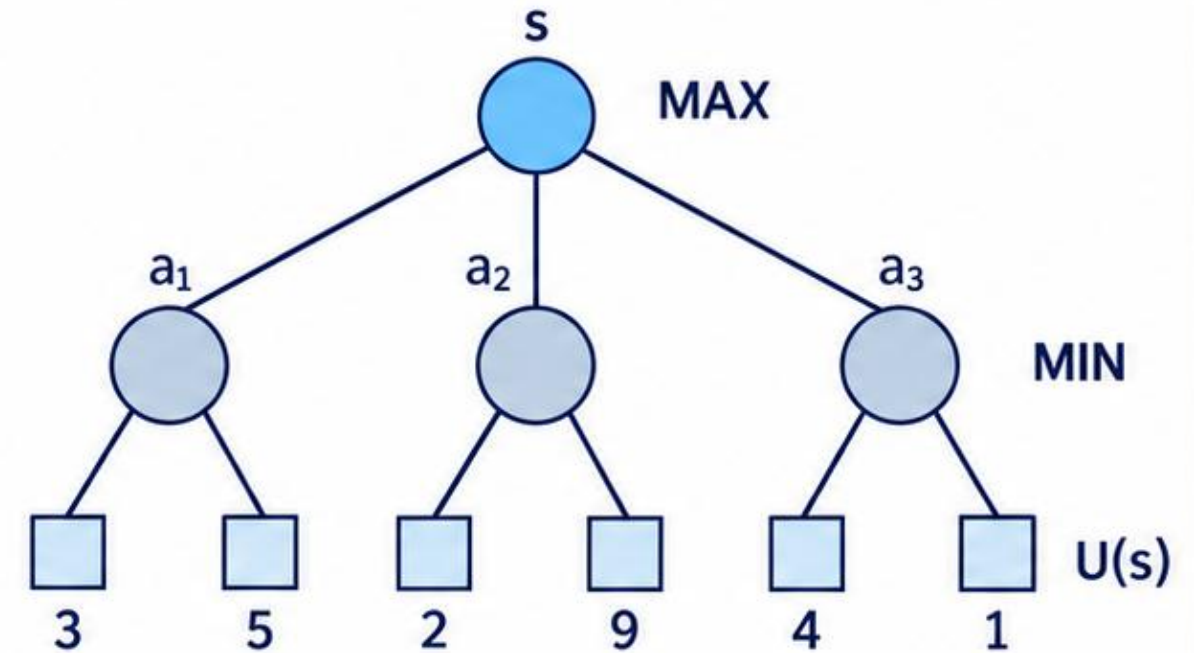
Minimax: definição recursiva

Calcular um valor para cada estado e selecionar uma ação na raiz.

Definição recursiva do valor de um estado

$$V(s) = \begin{cases} U(s) & \text{se } s \text{ é terminal} \\ \max_{a \in A(s)} V(\text{resultado}(s, a)) & \text{se } s \text{ não é terminal e joga MAX} \\ \min_{a \in A(s)} V(\text{resultado}(s, a)) & \text{se } s \text{ não é terminal e joga MIN} \end{cases}$$

Árvore de jogo (exemplo ilustrativo)



Escolha da ação na raiz MAX

$$a^* = \arg \max_{a \in A(s)} V(\text{resultado}(s, a))$$

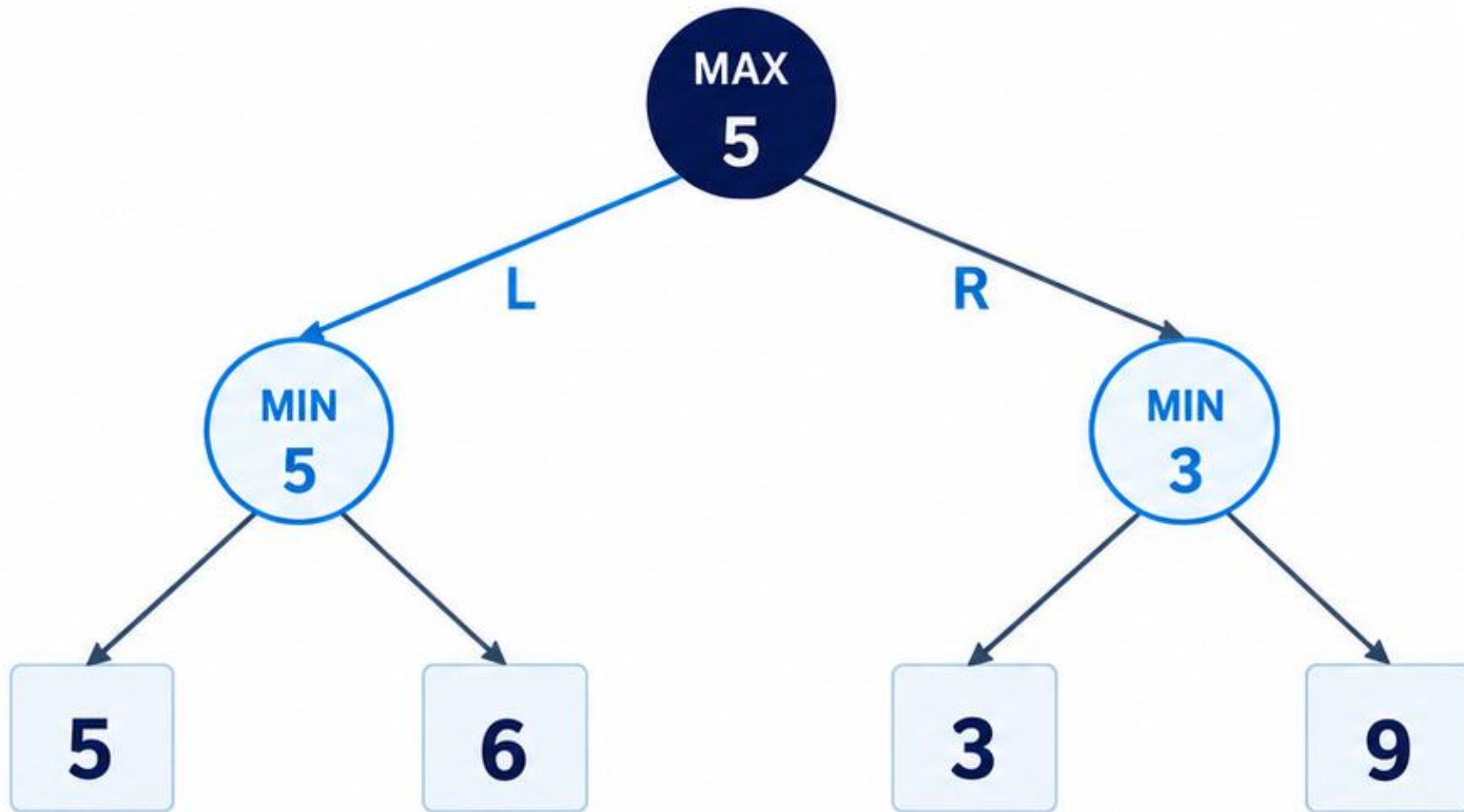
Notação e regras

V(s)	é um valor; a^* é uma ação. Não são o mesmo objeto.
A(s)	ações legais em s.
Empate	conservar a primeira ação.

Minimax: execução completa

Propagar os resultados de baixo para cima, alternando mínimo e máximo.

Árvore de jogo



Propagação dos valores

Passo	Cálculo	Resultado
1	$V(L) = \min(5, 6)$	5
2	$V(R) = \min(3, 9)$	3
3	$V(s_0) = \max(5, 3)$	5

4 folhas avaliadas • 7 nós visitados



Conclusão

A melhor ação é L: garante o valor 5 contra as respostas previstas no modelo.

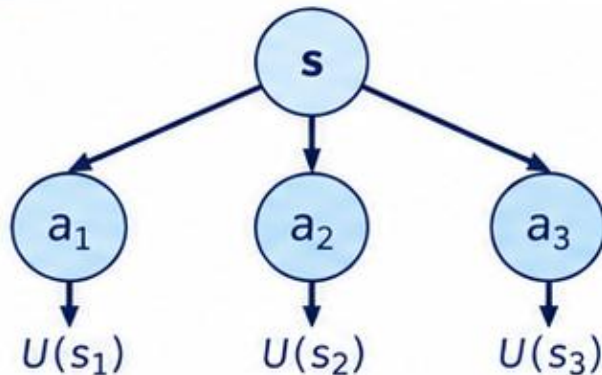
Do valor à ação: pseudocódigo Minimax

A recursão calcula valores; a função da raiz guarda a ação correspondente.

Valor recursivo (VALOR)

Calcula o valor minimax de um estado, com recursão sobre os sucessores.

```
01 VALOR(s):  
02   se terminal(s): devolver U(s)  
03   se jogador(s) = MAX:  $v \leftarrow -\infty$   
04   senão:  $v \leftarrow +\infty$   
05   para a em acoes(s):  
06      $r \leftarrow \text{VALOR}(\text{resultado}(s, a))$   
07     se jogador(s) = MAX:  
08        $v \leftarrow \max(v, r)$   
09     senão:  $v \leftarrow \min(v, r)$   
10   devolver v
```

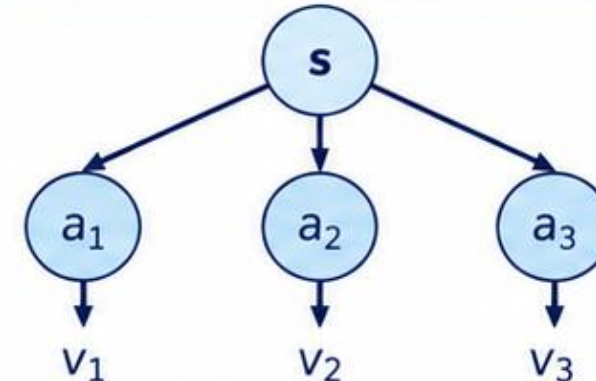


O valor de s é calculado recursivamente a partir dos valores dos sucessores, maximizando ou minimizando consoante o jogador.

Decisão na raiz MAX (DECIDIR)

Na raiz, avalia todas as ações e devolve a ação com melhor valor para o jogador MAX.

```
01 DECIDIR(s):  
02   melhor  $\leftarrow -\infty$ ; escolha  $\leftarrow$  nenhuma  
03   para a em acoes(s):  
04      $r \leftarrow \text{VALOR}(\text{resultado}(s, a))$   
05     se escolha = nenhuma  
06       ou  $r > \text{melhor}$ :  
07       melhor  $\leftarrow r$ ; escolha  $\leftarrow a$   
08   devolver escolha, melhor
```



Escolhe a ação com maior valor minimax e devolve essa ação e o respetivo valor.

Pré-condição de DECIDIR:

| s não terminal e jogador(s) = MAX.

Complexidade: quanto custa explorar?

Um nível adicional pode multiplicar o trabalho por aproximadamente **b**.

Modelo de custo

b: fator de ramificação

número de nós filhos por um nó

d: profundidade em ações

número de ações desde a raiz
até um nó no nível *d*

$$N_{\text{folhas}} = b^d$$

Total de nós (inclui a raiz)

$$N_{\text{total}} = \sum_{k=0}^d b^k$$

$$T_{\text{Minimax}} = O(b^d)$$

Árvore regular: $b = 3, d = 4$

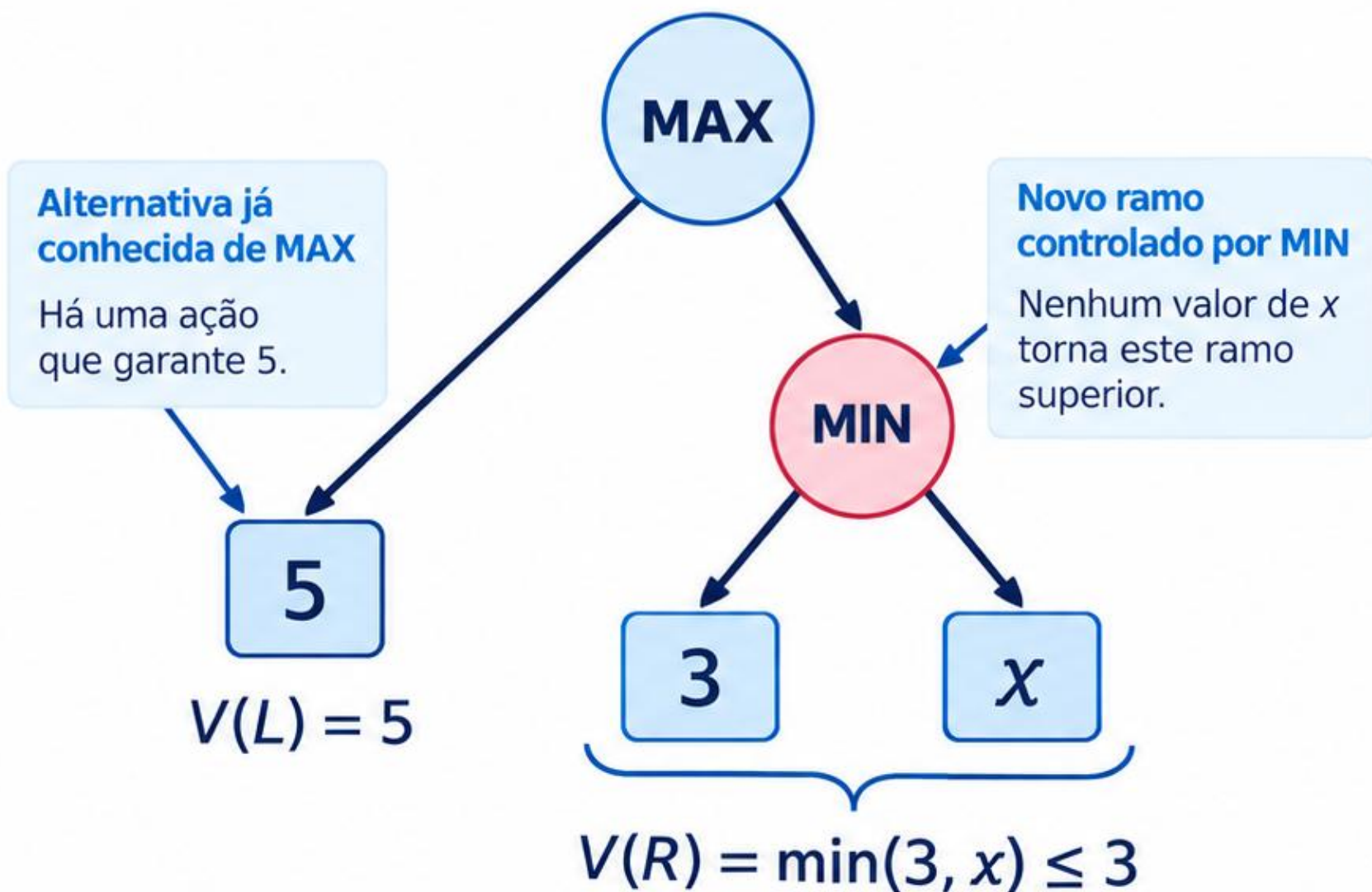
Nível	Nós no nível	Acumulado
0	1	1
1	3	4
2	9	13
3	27	40
4	81	121

Folhas (nível 4) = $3^4 = 81$

Total de nós = 121

Por que podemos ignorar um ramo?

Uma informação parcial pode ser suficiente para excluir uma alternativa.



1. Valor do ramo esquerdo (já conhecido)

$$V(L) = 5$$

Há uma ação que garante 5.

2. Ramo direito controlado por MIN

$$V(R) = \min(3, x) \leq 3$$

Nenhum valor de x torna este ramo superior.

3. Valor na raiz (MAX)

$$\max\{5, \min(3, x)\} = 5$$

x é a folha ainda não avaliada. Não precisamos de conhecer o seu valor.

**A poda preserva o valor da árvore considerada;
não é uma escolha aleatória.**

Alfa e beta: transmissão, atualização e corte

Cada chamada recebe os limites úteis dos seus antecessores.

α / limite associado a MAX

α representa a melhor avaliação (valor mínimo garantido) que MAX já pode alcançar ao longo do caminho de pesquisa.

$$\alpha \leftarrow \max(\alpha, v)$$

Alternativa já encontrada para MAX ao longo do caminho de pesquisa.

Atualiza-se em MAX; não diminui.

β / limite associado a MIN

β representa a melhor avaliação (valor máximo garantido) que MIN já pode alcançar ao longo do caminho de pesquisa.

$$\beta \leftarrow \min(\beta, v)$$

Alternativa já encontrada para MIN ao longo do caminho de pesquisa.

Atualiza-se em MIN; não aumenta.

Inicialização na raiz

$$\alpha = -\infty, \quad \beta = +\infty$$

Condição de corte (poda)

$$\alpha \geq \beta \Rightarrow \text{cortar}$$

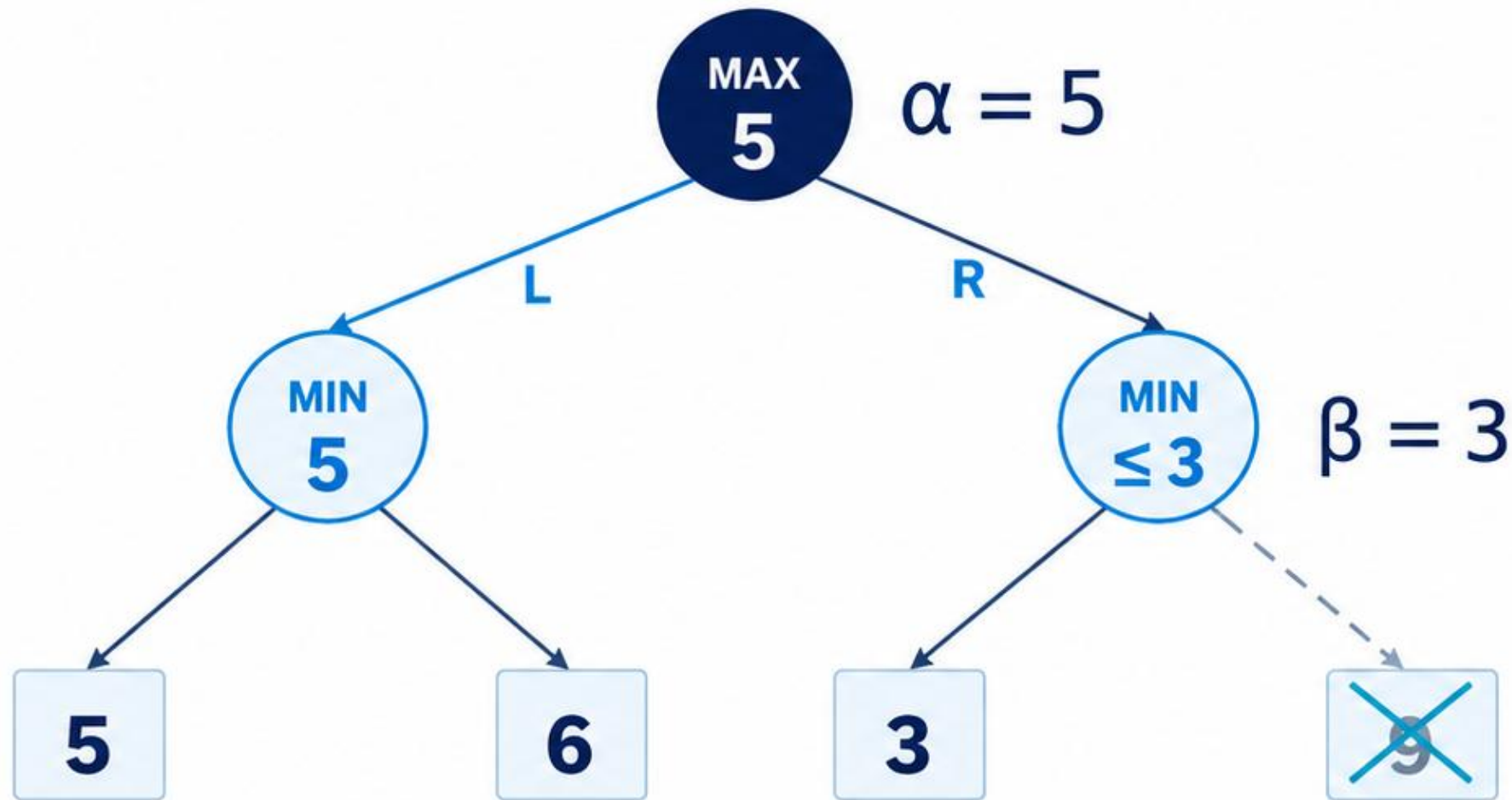
Os limites são locais a cada chamada, não variáveis globais entre ramos irmãos.

Cada chamada recebe o seu próprio par (α, β) herdado dos seus antecessores ao longo do caminho corrente da pesquisa.

Alfa-beta: execução na árvore conhecida

Percorrer da esquerda para a direita; comparar com a execução completa.

Árvore de procura (alfa-beta)



Registo da execução

- 1 MIN esquerdo**
 $\min(5, 6) = 5$
- 2 Raiz MAX**
 α passa de $-\infty$ para 5
- 3 MIN direito**
Vê 3: $\beta = 3$; recebe $\alpha = 5$
- 4 Corte**
 $5 \geq 3$: a folha 9 não é avaliada

Resultado da execução

3 folhas avaliadas • 6 nós visitados

Conclusão

O valor da raiz continua a ser 5 e a ação escolhida continua a ser **L**.

Cortes em nós MAX e em nós MIN

Exemplo didático adicional: três níveis de decisões, da esquerda para a direita.

1. Algoritmo MiniMax

Propaga os valores das folhas até à raiz, alternando entre nós MAX (escolhe o maior valor) e nós MIN (escolhe o menor valor).

2. Poda Alfa-Beta

Evita a exploração de ramos que não podem influenciar a decisão final, usando dois limites:

- α (alfa): melhor valor já encontrado para MAX (limite inferior).
- β (beta): melhor valor já encontrado para MIN (limite superior).

3. Regras de corte

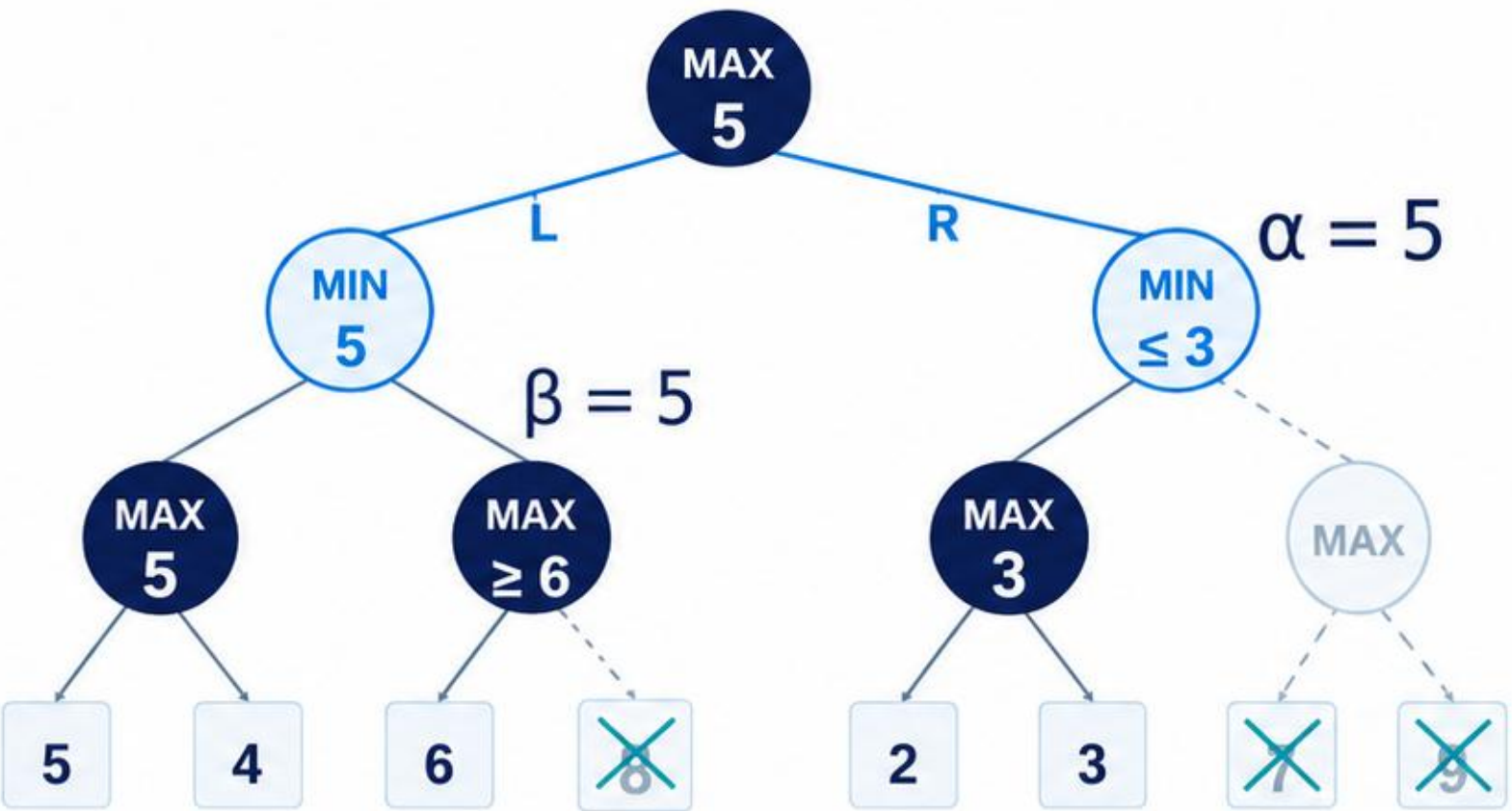
Corte em nó MAX

Se o valor de um filho $\geq \beta$, corta-se o restante dos filhos.

Corte em nó MIN

Se o valor de um filho $\leq \alpha$, corta-se o restante dos filhos.

4. Árvore de decisão com cortes alfa-beta



5. Exemplo de corte em nó MAX

Corte em MAX: $6 \geq \beta = 5$

No nó MAX (≥ 6), após avaliar o primeiro filho com valor 6, como $6 \geq \beta = 5$, o segundo filho (8) é podado.

6. Exemplo de corte em nó MIN

Corte em MIN: $\alpha = 5 \geq 3$

No nó MIN (≤ 3), após avaliar o primeiro filho com valor 3, como $\alpha = 5 \geq 3$, o segundo filho (7, 9) é podado.

7. Cálculo dos valores

- 1 Folhas: 5, 4, 6, 8, 2, 3, 7, 9 (8, 7 e 9 são podados).
- 2 $\text{MAX}(5, 4) = 5$
- 3 No nó MAX (≥ 6): primeiro filho 6 $\rightarrow 6 \geq \beta = 5$, corta o filho 8.
- 4 $\text{MIN}(5, \geq 6) = 5$
- 5 $\text{MAX}(2, 3) = 3$
- 6 No nó MIN (≤ 3): $3 \leq \alpha = 5$, corta o nó MAX (7, 9).
- 7 Valor final na raiz: $\text{MAX}(5, \leq 3) = 5$

8. Resultados da exploração

Valor minimax (raiz)	5
Total de nós na árvore	15
Nós visitados (avaliados)	11
Folhas avaliadas	5

Conclusão

Após o corte, o valor devolvido pode ser apenas um limite, não o valor exato do nó.

Pseudocódigo alfa-beta

A mesma recursão, com limites e uma condição de interrupção.

Função geral

$AB(s, \alpha, \beta)$: se $terminal(s)$: devolver $U(s)$

s – estado do jogo
 α – melhor valor já encontrado para o MAX (limite inferior)
 β – melhor valor já encontrado para o MIN (limite superior)
 $U(s)$ – valor utilidade do estado terminal s

Caso MAX

MAX procura maximizar o valor

```
01   $v \leftarrow -\infty$ 
02  para  $a$  em  $acoes(s)$ :
03       $r \leftarrow AB(resultado(s,a), \alpha, \beta)$ 
04       $v \leftarrow \max(v, r)$ 
05       $\alpha \leftarrow \max(\alpha, v)$ 
06      se  $\alpha \geq \beta$ : interromper
07  devolver  $v$ 
```

Caso MIN

MIN procura minimizar o valor

```
01   $v \leftarrow +\infty$ 
02  para  $a$  em  $acoes(s)$ :
03       $r \leftarrow AB(resultado(s,a), \alpha, \beta)$ 
04       $v \leftarrow \min(v, r)$ 
05       $\beta \leftarrow \min(\beta, v)$ 
06      se  $\alpha \geq \beta$ : interromper
07  devolver  $v$ 
```

Inicialização

Inicializar com $AB(s_0, -\infty, +\infty)$; guardar também a melhor ação na raiz.

Ordem de exploração e trabalho realizado

Manter a árvore e alterar apenas a ordem dos sucessores.

Comparação de ordens de exploração

Algoritmo	Ordem proposta	Folhas	Nós	Valor / ação
Minimax	L: 5, 6 → R: 3, 9	4	7	5 / L
Alfa-beta	L: 5, 6 → R: 3, 9	3	6	5 / L
Alfa-beta	L: 5, 6 → R: 9, 3	4	7	5 / L
Alfa-beta	R: 3, 9 → L: 5, 6	4	7	5 / L

MAX: experimentar primeiro ações promissoras.

MIN: encontrar cedo respostas de valor baixo.

Conclusão: Boa ordenação pode poupar avaliações; o custo de ordenar também deve ser medido.

Profundidade limitada: terminal ou corte?

Uma ação de MAX ou MIN consome uma unidade de profundidade.

Ideia principal

A investigação é limitada a uma profundidade máxima **d**. Quando a profundidade se esgota, os estados não terminais são avaliados pela função de avaliação **Eval**.

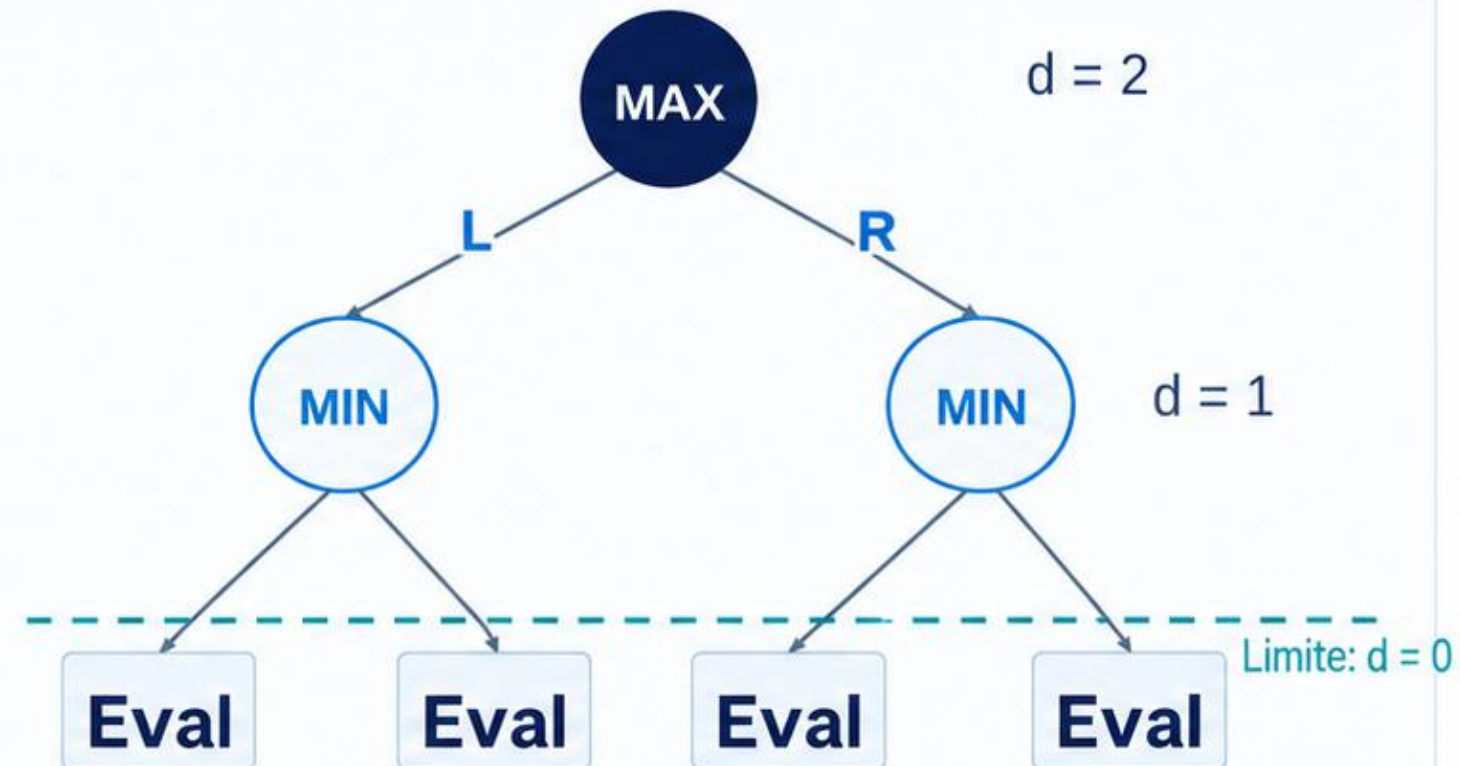
Procedimento (pseudocódigo)

```
01 VALOR(s, d):  
02   se terminal(s): devolver U(s)  
03   se d = 0: devolver Eval(s)  
04   explorar filhos com d - 1  
05   agregar por máximo ou mínimo
```

Cálculo recursivo

$$\text{VALOR}(s, d) = \begin{cases} U(s) & \text{se terminal}(s) \\ \text{Eval}(s) & \text{se } d = 0 \\ \max_{a \in A(s)} \text{VALOR}(\text{Result}(s, a), d - 1) & \text{se jogador}(s) = \text{MAX} \\ \min_{a \in A(s)} \text{VALOR}(\text{Result}(s, a), d - 1) & \text{se jogador}(s) = \text{MIN} \end{cases}$$

Exemplo de árvore de procura



Estados não terminais avaliados por estimativa.

Notas importantes

- **d é o número de ações ainda a explorar.**
- O teste terminal vem antes do teste $d = 0$.
- A poda preserva o valor da árvore limitada; Eval aproxima o jogo completo.

Construir uma avaliação: sinais e escalas

Exemplo portuário proposto: espera e risco penalizam; capacidade beneficia MAX.

Função de avaliação

Combinação ponderada de indicadores normalizados.

$$\text{Eval}(s) = -w_T \tilde{T}(s) - w_R \tilde{R}(s) + w_C \tilde{C}(s)$$

1 Espera: penalização

$$\tilde{T} = T/T_{\text{ref}}$$

$0 \leq T \leq T_{\text{ref}}$; $T_{\text{ref}} > 0$
Referência temporal fixa.



2 Risco: penalização

$$\tilde{R} = R \in [0, 1]$$

R é uma probabilidade.
Já está na escala $[0, 1]$.



3 Capacidade: benefício

$$\tilde{C} = C/C_{\text{ref}}$$

$0 \leq C \leq C_{\text{ref}}$; $C_{\text{ref}} > 0$
Referência de capacidade fixa.



Convenção didática

Pesos não negativos, com soma 1.

Procedimento

Definir primeiro o significado e a escala dos indicadores;
depois escolher os pesos.

Calcular a avaliação e analisar os pesos

Manter os estados; alterar apenas as prioridades expressas pelos pesos.

1. Indicadores normalizados

Estado	Espera $\tilde{T}$	Risco $\tilde{R}$	Capacidade $\tilde{C}$
s_1	0,2	0,4	0,8
s_2	0,5	0,2	0,6

2. Função de avaliação

$$\text{Eval}(s) = -w_T \tilde{T}(s) - w_R \tilde{R}(s) + w_C \tilde{C}(s)$$

Exemplo de cálculo para s_1 (pesos 0,5; 0,3; 0,2)

$$\begin{aligned}\text{Eval}(s_1) &= -0,5(0,2) - 0,3(0,4) + 0,2(0,8) \\ &= -0,06\end{aligned}$$

3. Avaliação com diferentes pesos

Pesos (w_T, w_R, w_C)	Eval(s_1)	Eval(s_2)	Preferência
(0,5; 0,3; 0,2)	-0,06	-0,19	s_1
(0,2; 0,6; 0,2)	-0,12	-0,10	s_2

Conclusão

Uma alteração de pesos pode inverter a preferência. Calibrar Eval na escala de U.

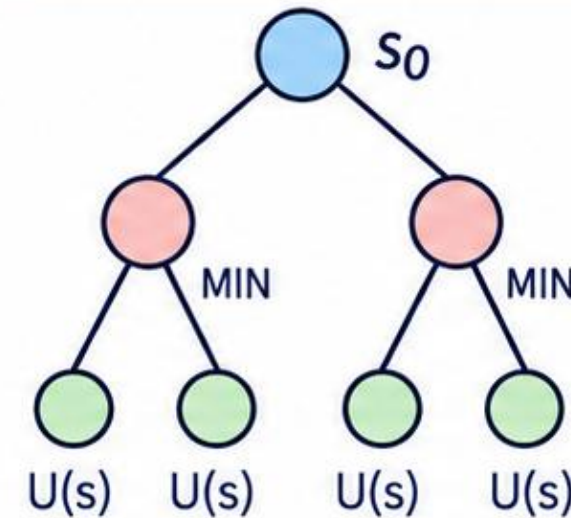
Como verificar uma implementação?

Usar árvores pequenas com resultados conhecidos antes de medir desempenho.

Teste	Comportamento esperado
Estado inicial terminal	Devolver $U(s)$, sem sucessores
Ações empatadas	Conservar a primeira ação
$d = 0$, não terminal	Usar $Eval(s)$
Terminal no limite	Usar $U(s)$, não $Eval(s)$

Igualdade a verificar na raiz

$$V_{\alpha\beta}(s_0) = V_{\text{Minimax}}(s_0)$$



Limites iniciais:
 $(-\infty, +\infty)$.
Mesma árvore,
profundidade
e função de avaliação.

Medições independentes



Folhas avaliadas



Nós visitados



Tempo de execução



Regra de implementação: Criar sucessores independentes; não alterar o estado de outros ramos.

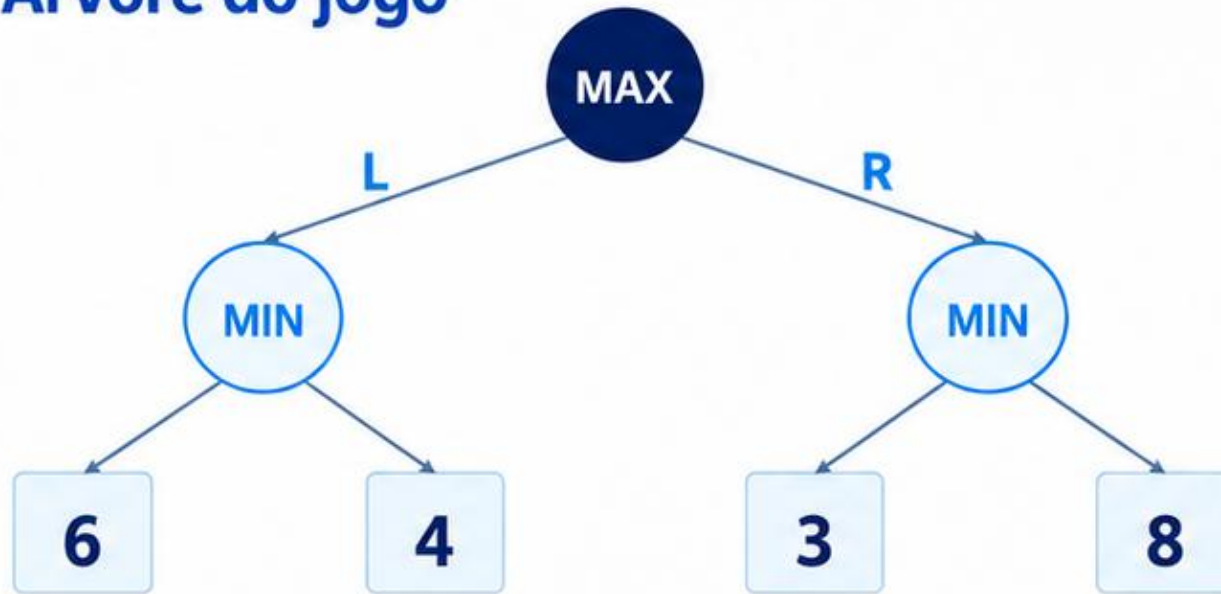


Testar o valor na raiz; não exigir valores exatos em todos os nós podados.

Exercício integrado: calcular, podar e comparar

Percorrer da esquerda para a direita; iniciar $\alpha = -\infty$ e $\beta = +\infty$.

1 Árvore do jogo



2 Cálculo dos valores

Nós MIN

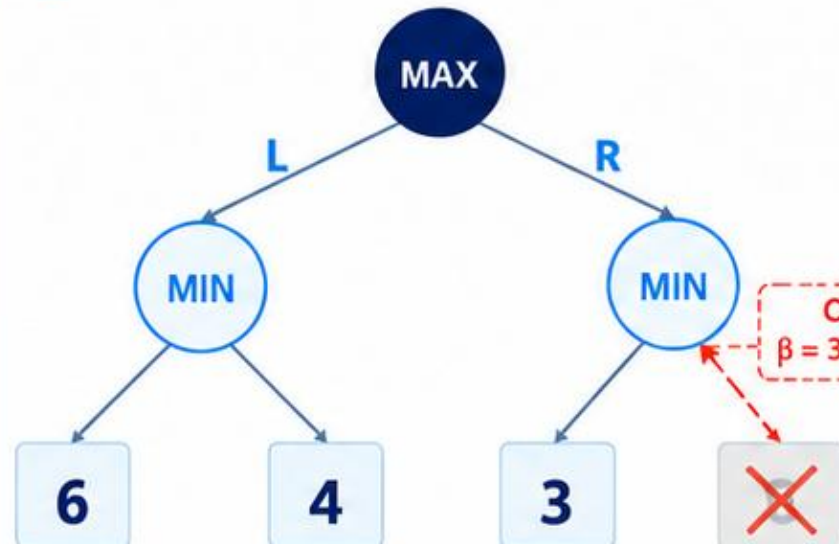
- Esquerda (L): $\min(6, 4) = 4$
- Direita (R): $\min(3, 8) = 3$

Raiz MAX

- $\max(4, 3) = 4$

Valor da raiz (MAX) = 4

4 Corte alfa-beta (ordem 3, 8)



Como ocorre o corte

- Após avaliar L, a raiz tem $\alpha = 4$.
- Em R (MIN), avalia-se a folha 3, ficando $\beta = 3$.
- Como $\beta = 3 \leq \alpha = 4$, a folha 8 não é avaliada (poda alfa-beta).

Resultado (ordem 3, 8)

- Ação escolhida na raiz: L (valor 4)
- Folhas avaliadas: 3 (6, 4, 3)
- Nós visitados: 6

5 Comparação de trabalho

Ordem em R	Folhas avaliadas	Nós visitados	Poda	Resultado
3, 8	3	6	Sim (poda 8)	4
8, 3	4	7	Não	4

Ordem 8, 3 (sem poupança)

- Em R (MIN), avalia-se primeiro 8, ficando $\beta = 8$.
- Como $\beta = 8 > \alpha = 4$, não há corte.
- Depois avalia-se 3 e obtém-se $\min(8, 3) = 3$.
- Folhas avaliadas: 4 (6, 4, 8, 3); nós visitados: 7.

6 Quatro tarefas

- Calcular os valores**
Valor da raiz e ação L ou R.
- Registrar os limites**
Atualizações de α e β .
- Justificar o corte**
Qual a folha não avaliada?
- Comparar o trabalho**
Folhas e nós, com e sem poda.

Desafio

Trocar a ordem 3, 8 por 8, 3.
O que muda e o que se mantém?

Síntese: decisão, otimização e aproximação



MINIMAX

- Calcula o valor de cada estado contra a melhor resposta do adversário.
- Estado terminal: usar $U(s)$.
- Estado MAX: escolher o maior valor dos sucessores.
- Estado MIN: escolher o menor valor dos sucessores.
- Na raiz MAX, guardar a ação com o maior valor.



ALFA-BETA

- Evita explorar ramos que já não podem alterar a decisão.
- Atualizar α em MAX e β em MIN.
- $\alpha \geq \beta \rightarrow$ interromper os restantes sucessores.
- Com limites iniciais $(-\infty, +\infty)$, preserva o valor Minimax da árvore considerada.
- A ordenação pode reduzir o trabalho; o ganho depende da árvore.



LIMITE + EVAL

- Limita a profundidade e estima estados ainda não terminais.
- Testar primeiro se o estado é terminal: devolver $U(s)$.
- Só se não for terminal e $d = 0$: devolver $Eval(s)$.
- Nos restantes casos, explorar com $d - 1$.
- Não garante o valor exato do jogo completo.

Representar



calcular



podar



avaliar



verificar

Comparar resultados, folhas avaliadas, nós visitados e tempo de execução.